Math4AI Capstone Report Title

Murad Taghizada

Yasin Veliyev

Nicat Alaskarli

Abstract

This project investigates the central question: *when does a one-hidden-layer nonlinear classifier genuinely improve upon a linear decision rule, and when is additional model complexity unnecessary?* To answer this, we compare softmax regression as a linear baseline with a one-hidden-layer neural network across three datasets with distinct geometric properties: a linearly separable Gaussian dataset, a nonlinear moons dataset, and the handwritten digits benchmark.

The results reveal a clear pattern. When the data is approximately linearly separable, both models achieve nearly identical performance, showing that additional nonlinear capacity offers no advantage. However, for intrinsically nonlinear data, the neural network delivers a noticeable improvement by learning curved decision boundaries that a linear model cannot represent. On the digits dataset, the neural network consistently outperforms the linear baseline, although the gain remains moderate, indicating the presence of both linear and nonlinear structure.

Further analyses—including capacity ablations, optimizer comparisons, failure-case investigations, and repeated-seed statistics—highlight how performance is shaped by the interaction between model capacity, optimization behavior, and data geometry. Overall, the findings demonstrate that nonlinear models are most valuable when the underlying structure cannot be captured by linear separation, while simpler linear models remain both sufficient and preferable when additional complexity does not yield meaningful benefits.

Introduction

In modern machine learning, a key modeling challenge is deciding whether a simple linear classifier is sufficient or whether more complex nonlinear structures are needed to capture the patterns present in the data. This project explores the central question: under what conditions does a one-hidden-layer nonlinear classifier offer meaningful improvements over a linear decision rule, and when does additional model complexity become unnecessary?

This question is scientifically important because it is closely tied to the geometric properties of data and the expressive capacity of different models. Linear approaches, such as softmax regression, rely on the assumption that classes can be separated by hyperplanes in the input space. While this assumption results in efficient and interpretable models, it breaks down when the true decision boundary is nonlinear. In contrast, introducing even a single hidden layer allows neural networks to apply nonlinear transformations that reshape the input space, making it possible to learn more complex decision boundaries.

Determining when this added flexibility is required is essential for both theoretical understanding and practical applications. Models that are too complex may overfit, demand larger datasets, and incur higher computational costs, whereas overly simple models may fail to capture important structure and underfit the data. By systematically comparing softmax regression with a one-hidden-layer neural network across datasets with different geometric characteristics, this project aims to provide both empirical and mathematical insight into the trade-off between simplicity and expressive power in model design.

Background and Mathematical Setup

We study supervised classification problems where inputs $x \in \mathbb{R}^d$ are assigned to discrete labels $y \in \{1, \dots, K\}$. The project compares two model families: softmax regression as a linear probabilistic classifier and a one-hidden-layer neural network as a nonlinear extension of this approach.

In **softmax regression**, class probabilities are defined as:

$$P(y = k \mid x) = \frac{\exp\,(w_k^\top x + b_k)}{\sum_{j=1}^{K} \exp\,(w_j^\top x + b_j)}$$

This formulation guarantees that predicted values are valid probabilities, as they are non-negative and sum to one. For a dataset $\{(x_i, y_i)\}_{i=1}^{n}$, the likelihood is:

$$\prod_{i=1}^{n} P(y_i \mid x_i)$$

Taking the logarithm yields:

$$\sum_{i=1}^{n} \log\, P(y_i \mid x_i)$$

Using one-hot encoding y_{ik}, this expression can be rewritten as:

$$\sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log\, P(y = k \mid x_i)$$

Minimizing the negative of this quantity results in the cross-entropy loss:

$$L = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik} \log\ P(y = k \mid x_i)$$

which shows that cross-entropy directly corresponds to the negative log-likelihood of the model.

Backpropagation provides an efficient way to compute gradients by systematically applying the chain rule. Instead of evaluating derivatives independently for each parameter, the method propagates error signals backward through the network, reusing intermediate computations from the forward pass. This makes training multi-layer models computationally practical.

The **one-hidden-layer neural network** extends the linear model by introducing a nonlinear transformation:

$$h = \tanh\,(W_1 x + b_1), \hat{y} = \text{softmax}(W_2 h + b_2)$$

This additional layer enables the model to represent nonlinear decision boundaries while still maintaining probabilistic outputs through the softmax function.

The parameter gradients are obtained via the chain rule. Defining $Z_2 = W_2 h + b_2$, we have:

$$\frac{\partial L}{\partial Z_2} = \hat{Y} - Y$$

which leads to:

$$\frac{\partial L}{\partial W_2} = (\hat{Y} - Y)^\top h, \frac{\partial L}{\partial b_2} = \sum(\hat{Y} - Y)$$

For the hidden layer, with $h = \tanh\,(Z_1)$and $Z_1 = W_1 x + b_1$:

$$\frac{\partial L}{\partial Z_1} = \left((\hat{Y} - Y)W_2\right) \odot (1 - h^2)$$

giving:

$$\frac{\partial L}{\partial W_1} = \left[\left((\hat{Y} - Y)W_2\right) \odot (1 - h^2)\right]^\top x$$

These expressions correspond directly to the gradients used during training.From a geometric viewpoint, softmax regression produces linear decision boundaries because it relies on affine transformations of the input space. This is well suited for the Gaussian dataset, where class distributions can be approximately separated by a hyperplane. In contrast, the moons dataset exhibits an intrinsically nonlinear structure that cannot be separated linearly. The hidden layer enables a transformation of the input space into a representation where such nonlinear boundaries become separable.

Methods

1. **Datasets.** Three datasets with distinct geometric characteristics are considered. The *linear Gaussian* dataset is synthetically constructed from two Gaussian distributions with identical covariance, resulting in an approximately linearly separable structure. The *moons* dataset is a commonly used nonlinear benchmark composed of two interleaving half-circles with added noise. The *digits* dataset is a real-world classification task consisting of handwritten digit images represented as normalized feature vectors.
2. Each dataset is divided into training, validation, and test subsets using stratified sampling to maintain class balance. The split proportions are fixed at 60% for training, 20% for validation, and 20% for testing. For the digits dataset, input features are scaled to the range $[0, 1]$to improve numerical stability during optimization.
3. **Models.** Two types of models are examined. Softmax regression acts as the linear baseline, mapping inputs to class probabilities through an affine transformation followed by a softmax function. The neural network extends this approach by introducing a single hidden layer with a tanh activation, followed by a softmax output layer. Unless specified otherwise, the hidden layer size is set to 32. Both models are optimized using cross-entropy loss with L2 regularization.
4. **Training Procedure.** Training is performed using mini-batch gradient descent with a batch size of 64. Softmax regression is optimized using standard gradient descent. For the neural network, three optimization methods are compared: stochastic gradient descent (SGD), momentum, and Adam. Default learning rates are applied unless explicitly stated, and all models are trained for a fixed number of epochs. The training data is reshuffled at the start of each epoch to support better generalization.
5. **Validation and Model Selection.** The validation set is used solely for selecting model configurations. In particular, it determines the optimal number of training epochs and fixes the final model setup. After this selection process, the test set is evaluated exactly once, ensuring a strict separation between model tuning and final performance measurement.
6. **Evaluation Metrics.** Performance is assessed using classification accuracy and cross-entropy loss. For low-dimensional datasets, decision boundary plots are used to visualize model behavior. For the digits dataset, additional analysis includes per-class accuracy and repeated-seed evaluation to measure performance stability across different random initializations.

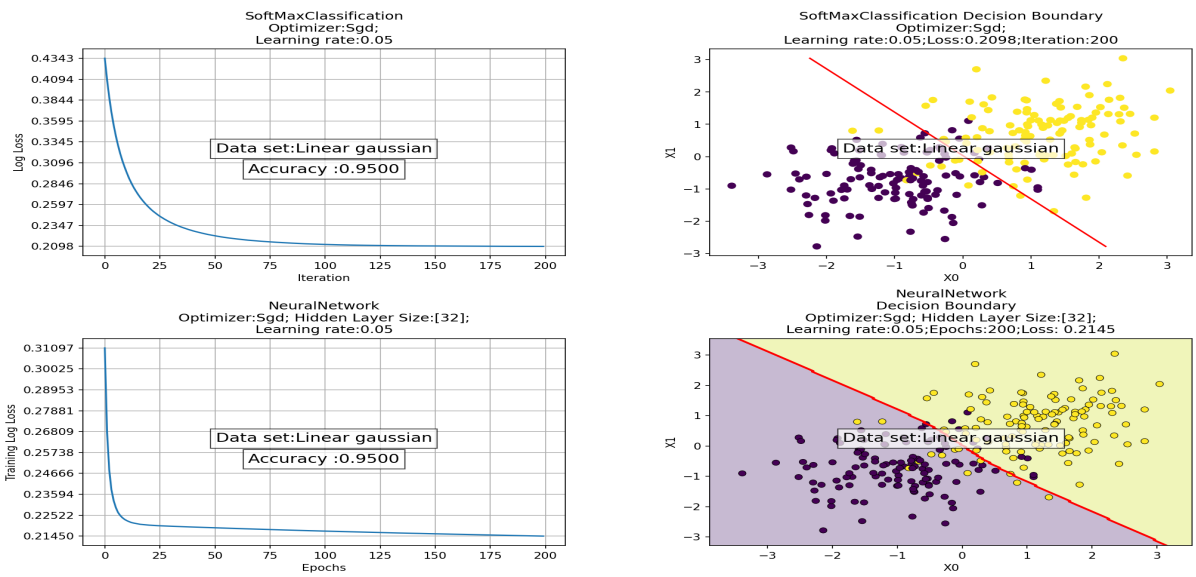
Implementation Sanity Checks

1. Gradient check was performed on a small subset of the data by comparing analytically computed gradients with their numerical approximations. The observed relative differences were extremely small, around $2.46 \times 10^{-8}$for softmax regression and $4.33 \times 10^{-8}$for the neural network, providing strong evidence that the backpropagation procedure was implemented correctly.
2. We confirmed that the predicted class probabilities from both models sum to one, verifying that the softmax outputs represent valid probability distributions.
3. Model behavior was evaluated on a very small subset of the training data. In this setting, both models exhibited a consistent decrease in training loss after a few optimization steps, and the neural network was able to overfit the subset, reaching nearly perfect training accuracy. This indicates that the models have sufficient representational capacity and that the optimization process is functioning properly.

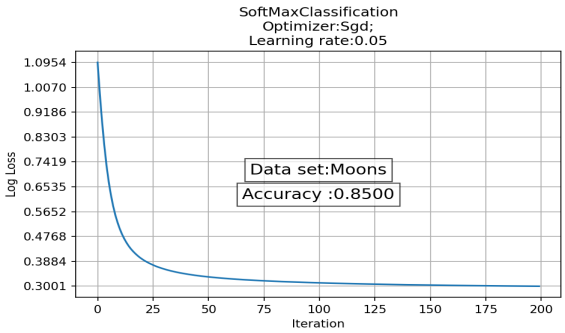
Finally, training was monitored to ensure numerical stability, confirming that no NaN or infinite values were produced during the optimization process.

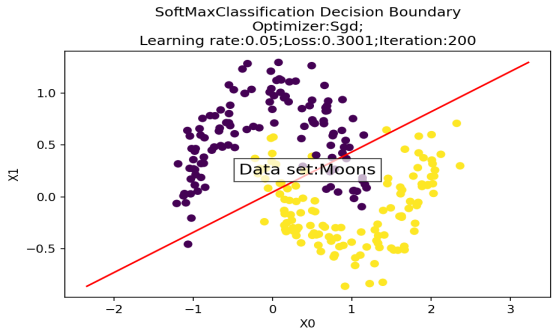
Experiments

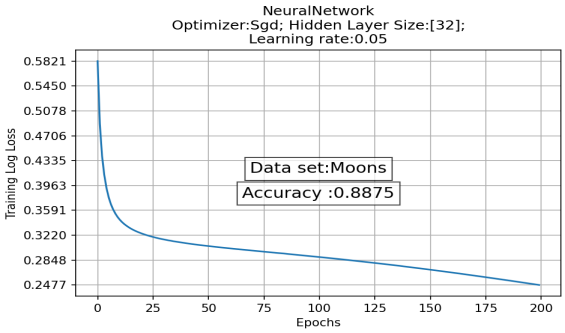
1.On the linear Gaussian dataset, both softmax regression and the neural network achieve similar performance (≈0.95 accuracy) and learn nearly identical linear decision boundaries. This indicates that a linear model is sufficient in this setting, and additional model complexity does not provide benefits.

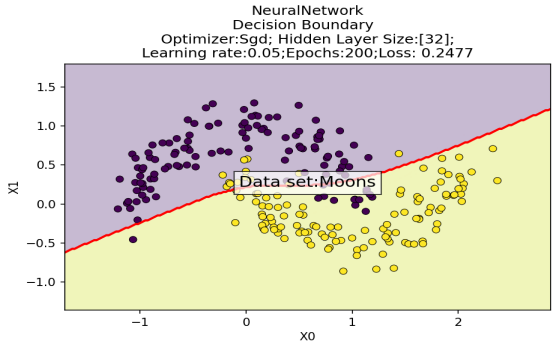


2.On the moons dataset, a clear difference emerges. Softmax regression achieves about 0.85 accuracy and underfits due to its linear nature, while the neural network improves performance to approximately 0.8875 by learning a nonlinear decision boundary.

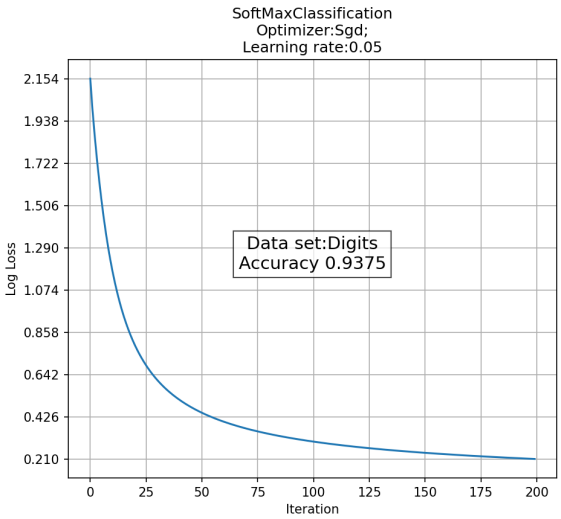


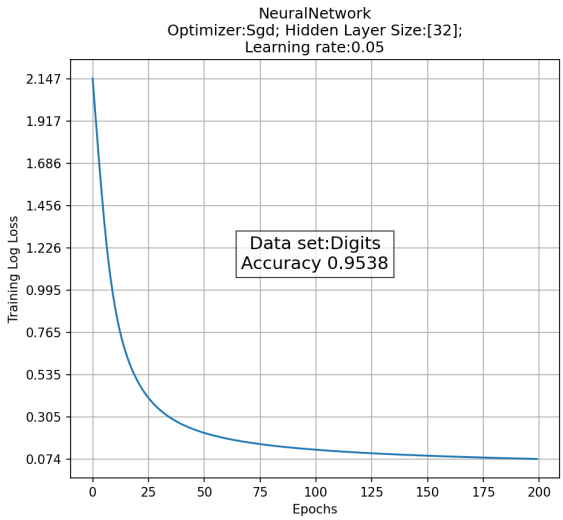




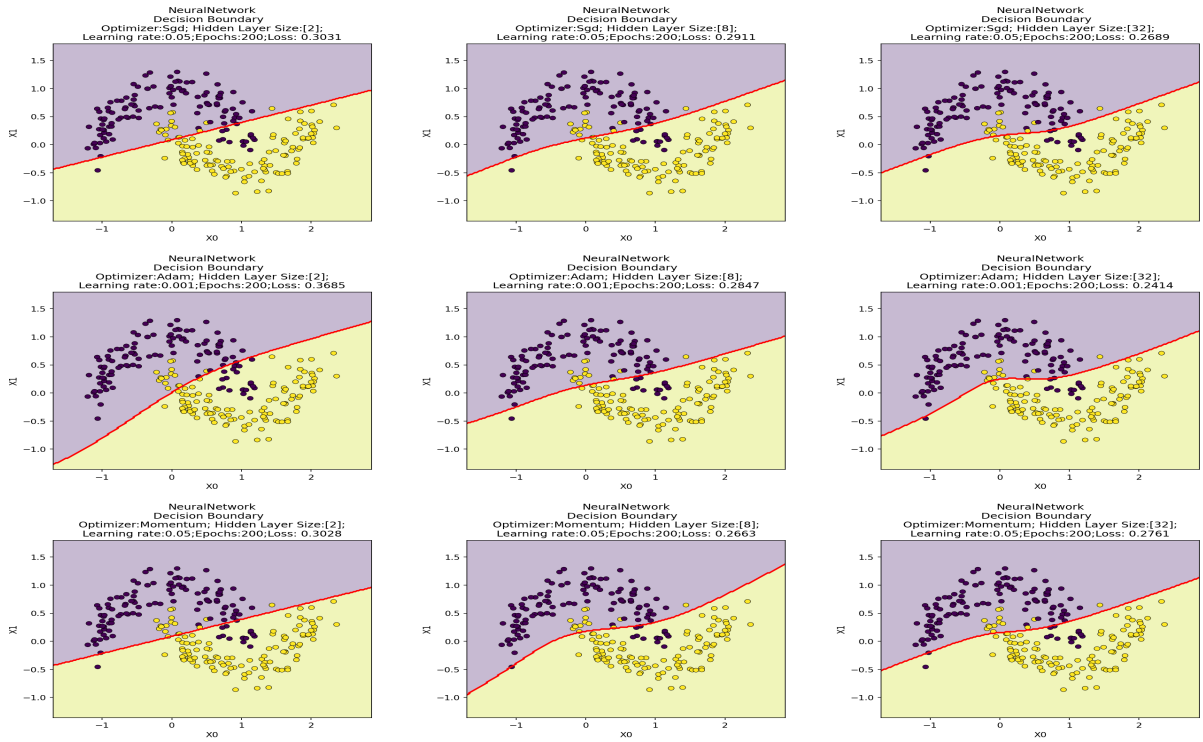


3.For the digits dataset, softmax regression reaches 0.9375 accuracy, while the neural network achieves 0.9538. The improvement is modest but consistent, suggesting that the data contains both linear and nonlinear structure.

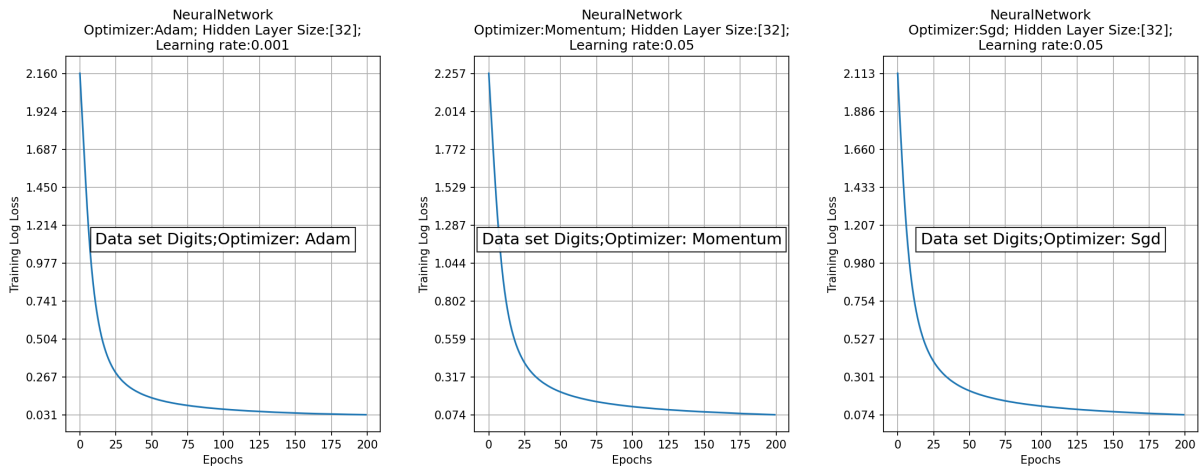




4.Capacity Ablation. On the moons dataset, increasing the hidden layer width improves performance. A width of 2 leads to underfitting, while widths of 8 and 32 allow the model to better capture the nonlinear structure, although gains diminish as capacity increases.



5.Optimizer Study. On the digits dataset, Adam converges faster and achieves lower loss compared to momentum and SGD, while SGD converges more slowly. This highlights the role of optimization methods in determining training efficiency under fixed conditions.

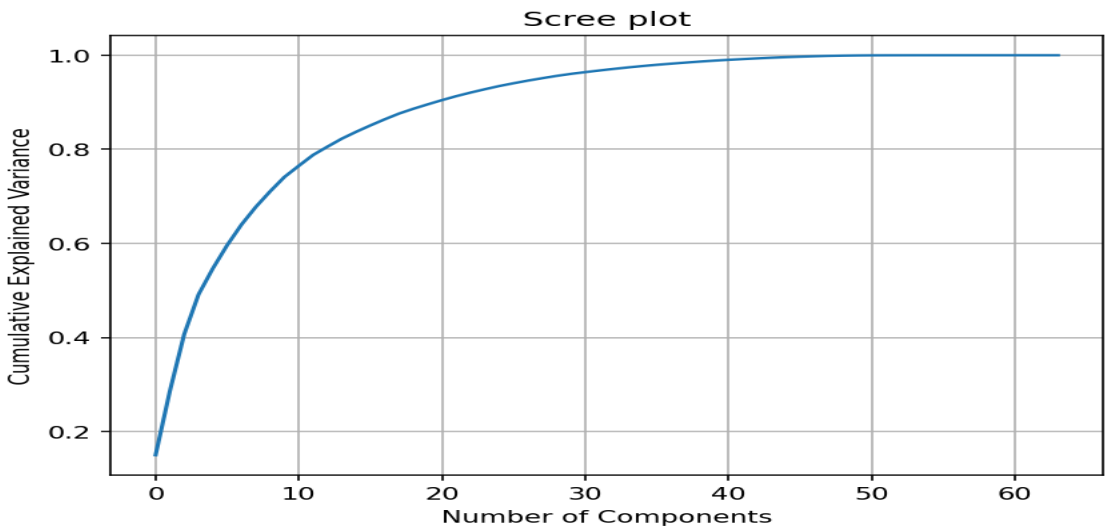


6.Failure Case Analysis. Class imbalance leads to biased predictions toward the majority class, reducing performance. Additionally, training on a very small dataset causes strong overfitting in the neural network, with high training accuracy but poor generalization, demonstrating the risks of excessive capacity with limited data.

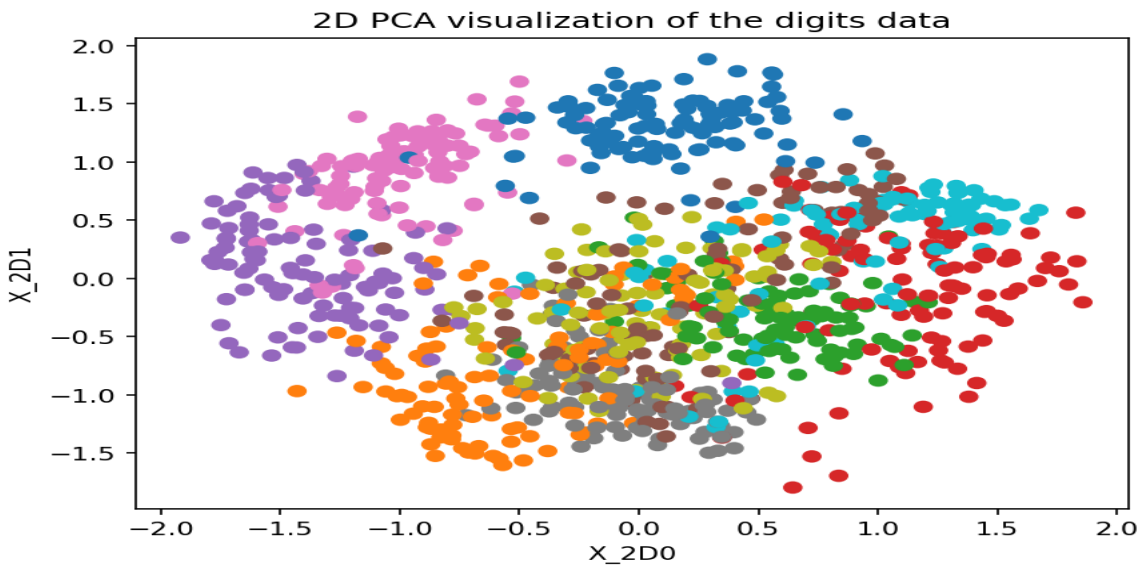
7.Repeated-Seed Statistics. Across five runs, softmax regression achieves a mean accuracy of 0.93478 and mean loss of 0.20635, with extremely narrow confidence intervals. This indicates highly stable performance across random initializations and strengthens confidence in the reported results.

Advanced Track A

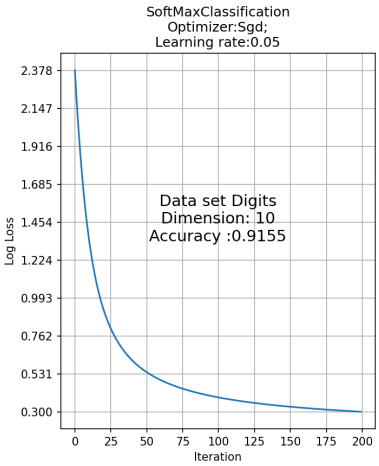
PCA is a linear technique that transforms data into a set of orthogonal directions ordered by variance, enabling high-dimensional inputs to be approximated in a lower-dimensional space while retaining most of the important information.A scree plot was used to examine how variance is distributed across principal components. The results indicate that a substantial portion of the total variance is captured within the first 20 to 40 components, suggesting that the digits dataset contains considerable redundancy and can be compressed effectively with limited loss of information.

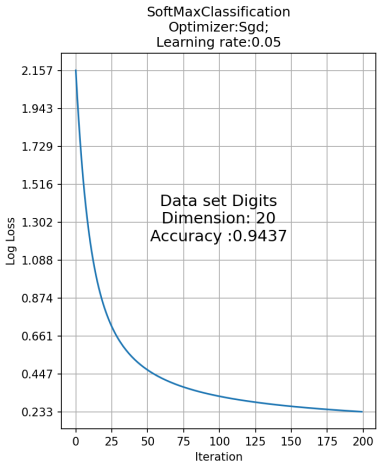


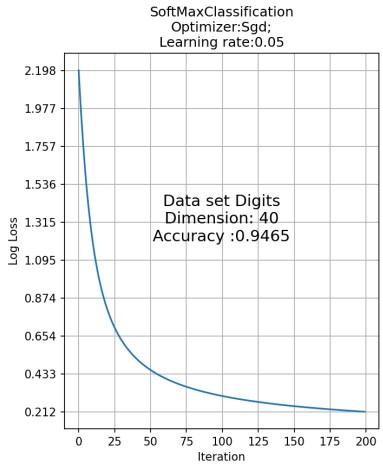
To gain further insight into the structure of the data, a two-dimensional PCA projection was visualized. Although some separation between classes is observable, there remains significant overlap, indicating that the dataset is not linearly separable in such a low-dimensional representation. This demonstrates the limitations of purely linear projections when dealing with complex data distributions.



The effect of dimensionality reduction was further evaluated by training softmax regression on PCA-transformed data with dimensions $m \in \{10,20,40\}$. As the number of retained components increases, classification accuracy improves. With only 10 components, performance is noticeably lower due to loss of information. At 20 components, accuracy increases substantially, and with 40 components, it approaches the performance achieved using the full feature space, showing diminishing gains as dimensionality grows.







Overall, these results indicate that most relevant information in the digits dataset lies within a lower-dimensional linear subspace, but aggressive dimensionality reduction removes important structure. While PCA is effective for reducing dimensionality and improving efficiency, it does not fully capture nonlinear relationships, which explains why nonlinear models can still outperform purely linear approaches.

Discussion

The findings of this study provide a clear response to the central question: linear models are adequate when the data is close to linearly separable, whereas a one-hidden-layer neural network offers advantages when the data exhibits nonlinear structure.For the linear Gaussian dataset, both models achieve very similar performance and learn nearly identical linear decision boundaries. This confirms that when the underlying data geometry can be separated by a hyperplane, increasing model complexity does not yield additional benefits.

In contrast, the moons dataset reveals the limitations of linear classifiers. Softmax regression is unable to model the curved decision boundary and therefore underfits the data, while the neural network successfully captures the nonlinear structure. This demonstrates the key advantage of the hidden layer: it enables a transformation of the input space that makes nonlinear patterns representable.The digits dataset presents a more intermediate scenario. The neural network consistently outperforms softmax regression, but only by a modest margin. This suggests that the dataset contains both linear and nonlinear characteristics. While the hidden layer provides additional flexibility, much of the structure can still be explained by a linear model.

The ablation experiments further illustrate the impact of model capacity and optimization. Increasing the width of the hidden layer improves the network's ability to represent complex decision boundaries, although the gains become smaller as capacity continues to grow. In

addition, the optimizer comparison shows that training behavior depends strongly on the optimization method, with some optimizers achieving faster convergence than others.

The failure-case analysis highlights important constraints. When data is limited, high-capacity models tend to overfit, achieving very high training accuracy but poor generalization. Similarly, class imbalance can introduce bias toward majority classes, negatively affecting performance. These observations underline the importance of aligning model complexity with both the data distribution and the available sample size.Finally, the repeated-seed experiments indicate that the results are consistent across different random initializations. This provides additional confidence that the observed differences between models are robust and not driven by randomness.

Limitations

1. This study analyzes the behavior of a linear classifier (softmax regression) and a one-hidden-layer neural network from a geometric viewpoint. Softmax regression defines decision boundaries as hyperplanes in the input space, whereas the neural network applies nonlinear transformations that enable the formation of curved and more flexible boundaries. The experimental results across datasets with different geometric structures provide empirical support for these differences in representational capacity.
2. The main conclusions are grounded in evidence from several experiments. On the linear Gaussian dataset, both models achieve comparable performance, indicating that when the data is close to linearly separable, a linear model is sufficient. In contrast, the moons dataset highlights the limitations of linear decision rules: softmax regression underfits due to its inability to model curved boundaries, while the neural network successfully captures the nonlinear structure. For the digits dataset, the neural network consistently achieves slightly higher performance, suggesting that both linear and nonlinear components are present in the data.
3. These results show that the hidden layer alters the geometry of the problem by transforming the input space into a representation where nonlinear decision boundaries become separable. However, this benefit depends on the properties of the dataset and the availability of sufficient training data. The failure-case analysis further demonstrates that increasing model capacity can lead to overfitting, particularly when the data is limited or exhibits class imbalance.
4. Nevertheless, the scope of this study remains limited. Only a single neural network architecture with one hidden layer is considered, and more complex or deeper models may behave differently. In addition, the experiments rely on a small number of datasets, which may not fully reflect the variability found in real-world scenarios. Although repeated-seed experiments provide some indication of stability, the number of runs is relatively small.
5. Therefore, while the results clearly identify situations in which linear models are sufficient and cases where nonlinear models offer advantages, these conclusions are constrained by the specific models, datasets, and experimental design used in this study, and should not be generalized beyond these settings without further investigation.

These findings also indicate situations where increasing model complexity is not beneficial. When the data is close to linearly separable, such as in the Gaussian dataset, introducing additional complexity does not result in noticeable performance gains. Likewise, when the dataset is small, models with higher capacity are prone to overfitting, achieving very high training accuracy while failing to generalize well. In these scenarios, simpler linear models tend to offer more stable and efficient performance.

In addition, the repeated-seed experiments emphasize the value of evaluating models across multiple runs rather than relying on a single outcome. The minimal variation observed across different random initializations suggests that the performance differences between models are consistent and not caused by randomness. This strengthens the reliability of the results and supports the conclusion that the observed improvements are due to genuine model behavior rather than fluctuations in the training process

References

- Jiang, M., Liang, Y., Feng, X., Fan, X., Pei, Z., Xue, Y., & Guan, R. (2018). Text classification based on deep belief network and softmax regression. *Neural Computing and Applications*, *29*(1), 61-70.
- Ren, Y., Zhao, P., Sheng, Y., Yao, D., & Xu, Z. (2017, August). Robust softmax regression for multi-class classification with self-paced learning. In *Proceedings of the 26th international joint conference on artificial intelligence* (pp. 2641-2647).
- Stanford University. (n.d.). *Neural Networks: Case Study*. CS231n. Retrieved April 2, 2026, from https://cs231n.github.io/neural-networks-case-study/
- Stanford University. (n.d.). *Linear Classification*. CS231n. Retrieved April 2, 2026, from https://cs231n.github.io/linear-classify/
- Aman, R. (n.d.). *Training Neural Nets II: Gradient Checks*. Aman.ai / CS231n. Retrieved April 2, 2026, from https://aman.ai/cs231n/training-neural-nets-II/#gradient-checks
- Zhong, K., Song, Z., Jain, P., Bartlett, P. L., & Dhillon, I. S. (2017, July). Recovery guarantees for one-hidden-layer neural networks. In *International conference on machine learning* (pp. 4140-4149). PMLR.
- Huang, G. B., Chen, Y. Q., & Babri, H. A. (2000). Classification ability of single hidden layer feedforward neural networks. *IEEE transactions on neural networks*, *11*(3), 799-801.
- Mao, A., Mohri, M., & Zhong, Y. (2023, July). Cross-entropy loss functions: Theoretical analysis and applications. In *International conference on Machine learning* (pp. 23803-23828). pmlr.
- Ho, Y., & Wookey, S. (2019). The real-world-weight cross-entropy loss function: Modeling the costs of mislabeling. *IEEE access*, *8*, 4806-4813.
- Nguyen, T., Raghu, M., & Kornblith, S. (2020). Do wide and deep networks learn the same things? uncovering how neural network representations vary with width and depth. *arXiv preprint arXiv:2010.15327*.

- Soofi, A. A., & Awan, A. (2017). Classification techniques in machine learning: applications and issues. *Journal of basic & applied sciences*, *13*, 459-465.
- Kotsiantis, S. B., Zaharakis, I. D., & Pintelas, P. E. (2006). Machine learning: a review of classification and combining techniques. *Artificial Intelligence Review*, *26*(3), 159-190.
-

Appendix

This appendix provides additional experimental details and supporting results that complement the main analysis while maintaining the clarity of the main text.

Appendix A: Model Hyperparameters

The experiments were conducted using the following hyperparameter settings, applied consistently unless otherwise stated:

Model	Optimizer	Learning Rate	Batch Size	Epochs	Regularization (λ)
Softmax Regression	SGD	0.05	64	200	1×10^{-4}
Neural Network	Adam	0.001	64	200	1×10^{-4}
Neural Network	SGD	0.05	64	200	1×10^{-4}
Neural Network	Momentum	0.05	64	200	1×10^{-4}

All models were optimized using mini-batch gradient descent, with the training data reshuffled at each epoch to promote stable learning.

Appendix B: Data Splitting and Preprocessing

All datasets were partitioned using stratified sampling to maintain class balance across splits. The fixed split proportions were:

- 60% training
- 20% validation
- 20% test

For the digits dataset, input features were normalized to the range $[0, 1]$, ensuring numerical stability during optimization.

Appendix C: PCA Dimensionality Analysis

To further investigate the role of input dimensionality, softmax regression was evaluated on PCA-reduced representations of the digits dataset.

Figure A1 demonstrates how classification performance varies with the number of retained principal components.

- $m = 10$: noticeable degradation due to information loss
- $m = 20$: substantial recovery in performance
- $m = 40$: performance approaches that of the full-dimensional input

These results indicate that most informative structure resides in a lower-dimensional subspace, although overly aggressive compression removes critical information.

Appendix D: Additional Training Observations

Supplementary experiments provide further insight into training behavior:

- Loss curves confirm that Adam converges faster and more efficiently than SGD and momentum.
- Increasing hidden layer width improves the model's ability to represent nonlinear structure, though gains diminish at higher capacities.
- Experiments on limited data reveal that the neural network can overfit, achieving high training accuracy while failing to generalize effectively.

These patterns align closely with the trends discussed in the main experimental analysis.

Appendix E: Stability and Reproducibility

Repeated-seed experiments demonstrate that model performance remains highly consistent across different random initializations. The extremely narrow confidence intervals reported in the main results provide strong evidence that the observed performance differences are robust and not driven by randomness.